

Predictive Storage Caching and Tiered Data Migration

Nilay Sadanand Borkar: 2023A7PS0401G

Ishan Kaustubh Mehta: 2023A7PS0379G

Outline

1. Background
2. Simulation
3. Experiments
4. Next Steps
5. Questions



Background

Background

There is an upper bound on the amount of the Cache memory available to us, and this memory is not enough to hold all the KV Cache generated during the prefill phase.

There is some storage available in the GPU itself which is very fast however this is very expensive and can only hold a very small amount of cache.

Thus, we use tiered caching.

However there is still a problem in this, as there is a time cost associated with moving key value pairs from different tiers of storage to the GPU, where they will be used.

Also since cache fetching operations are reactive, meaning the cache is fetched from storage after the Model requests it, we need even more time.

Simulations

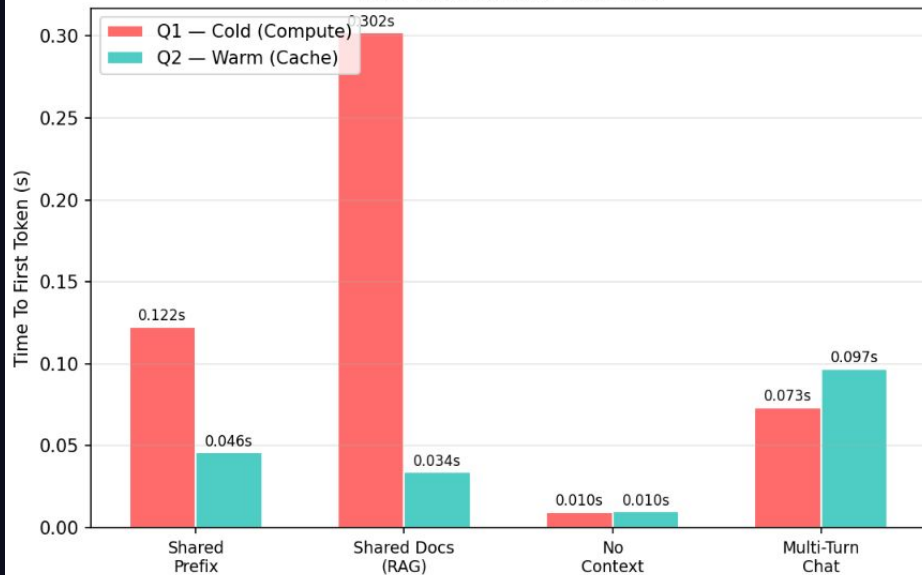
Simulations

First we simulated LMCache, a tiered storage caching system which uses, GPU, Memory and Storage to store key value pairs.

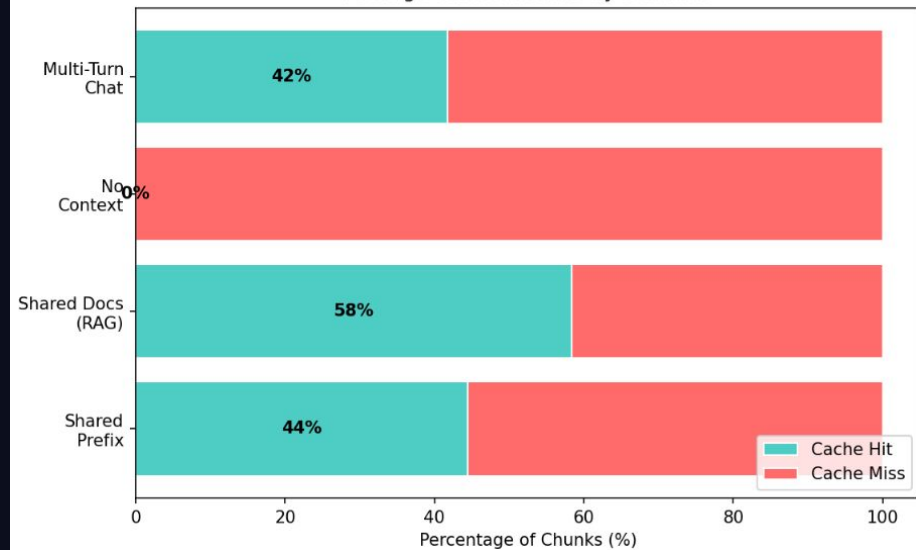
We wrote a script where we used some heuristic values to simulate how such a cache would perform during this.

Through this we wanted to know if tiered caching is of any help at all in user devices where compute power is limited.

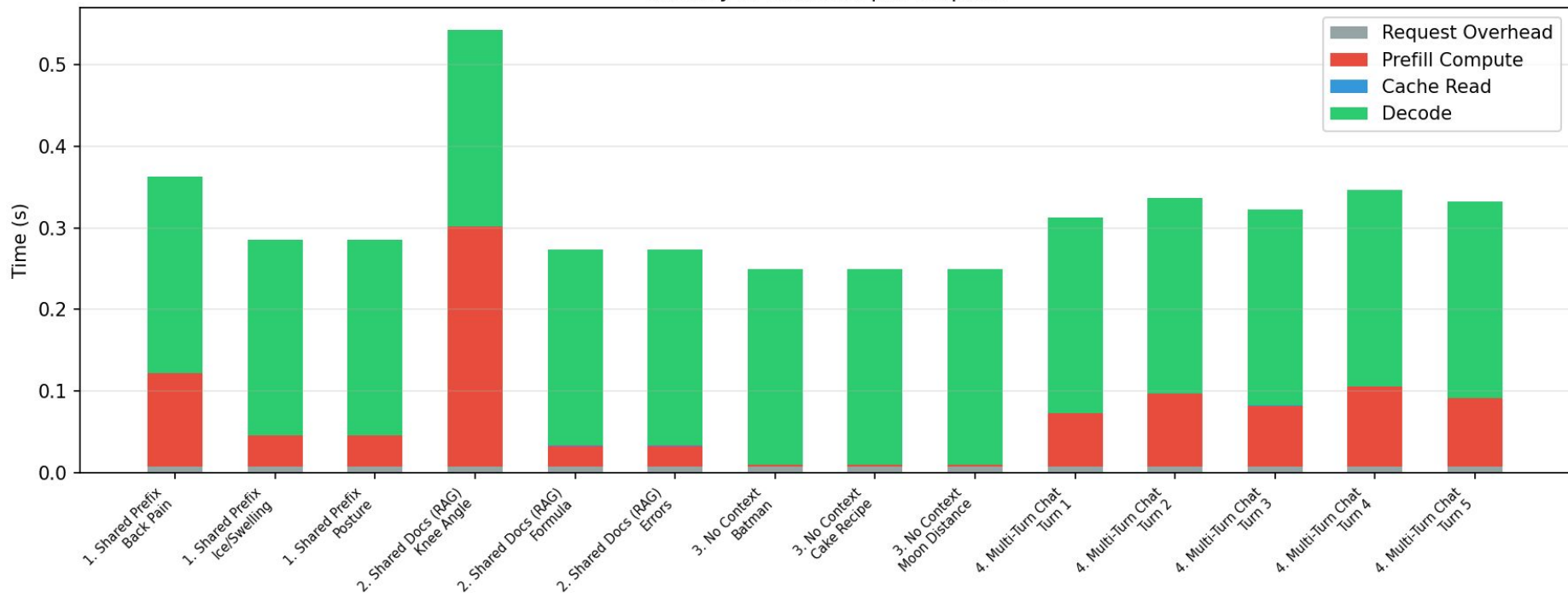
TTFT: Cold Start vs Cache Hit



Average Cache Hit Rate by Scenario



Latency Breakdown per Request



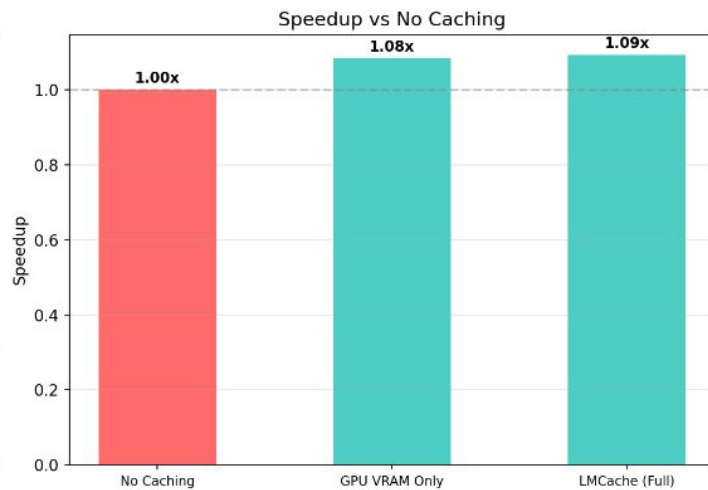
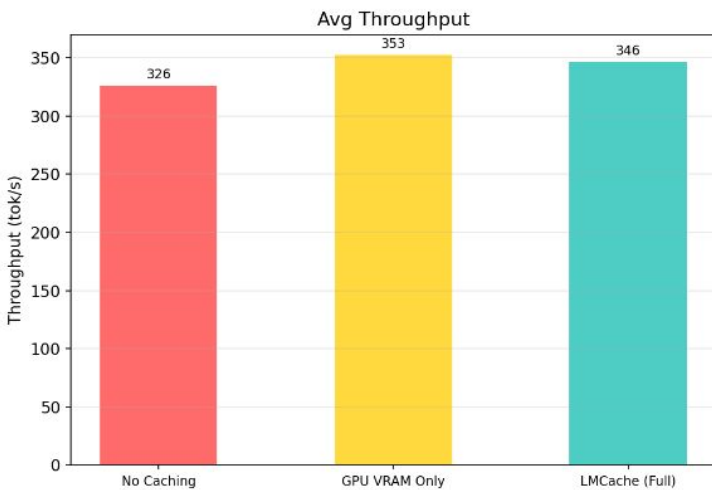
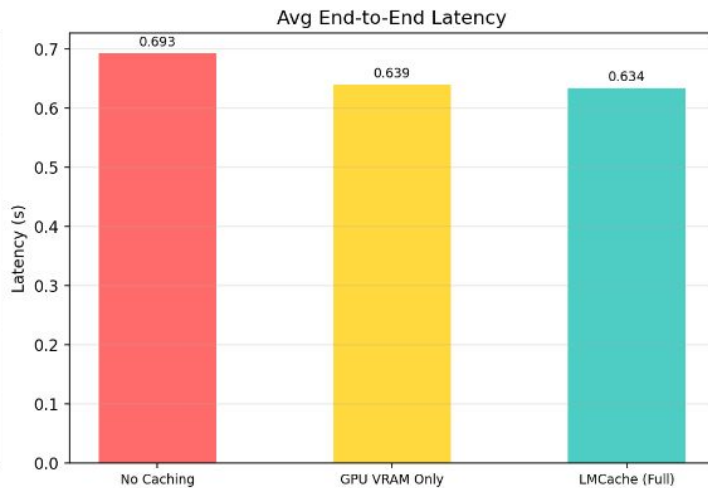
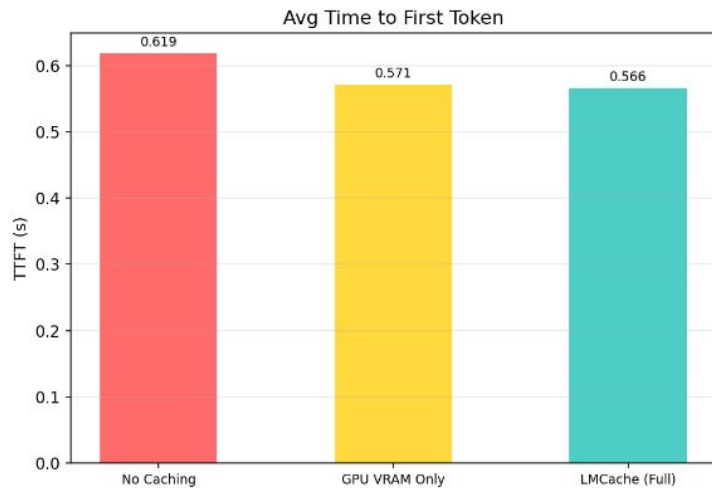
Experiments

Experiments

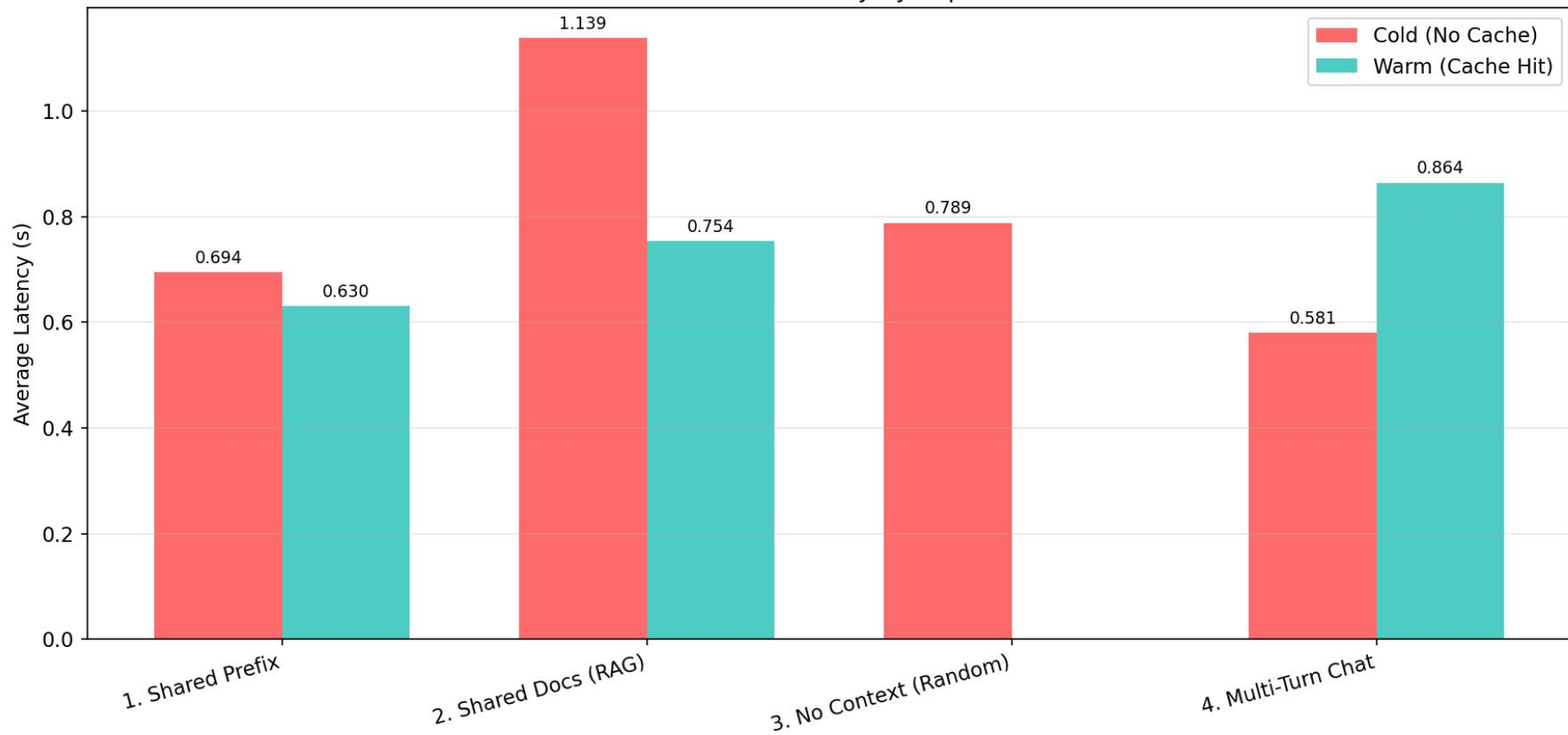
After the simulation, we wanted to test the LMCache model in reality

So we used the vLLM library to test it out.

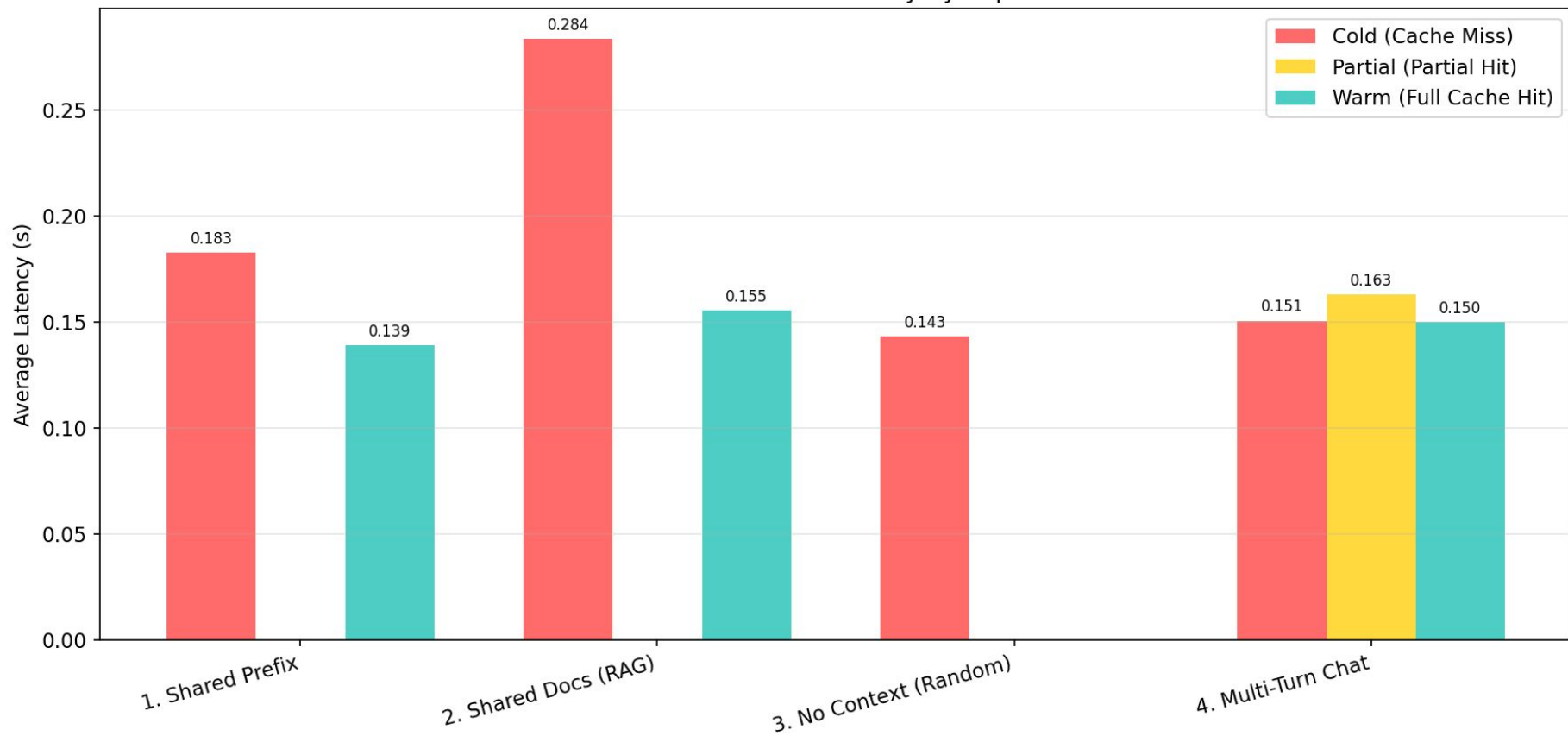
We tested QWEN 2.5 0.5B parameter model on 4 different different workloads (Prefix, RAG, No context, Multiturn chat) with 50 prompts each, to test out how LMCache would actually work.



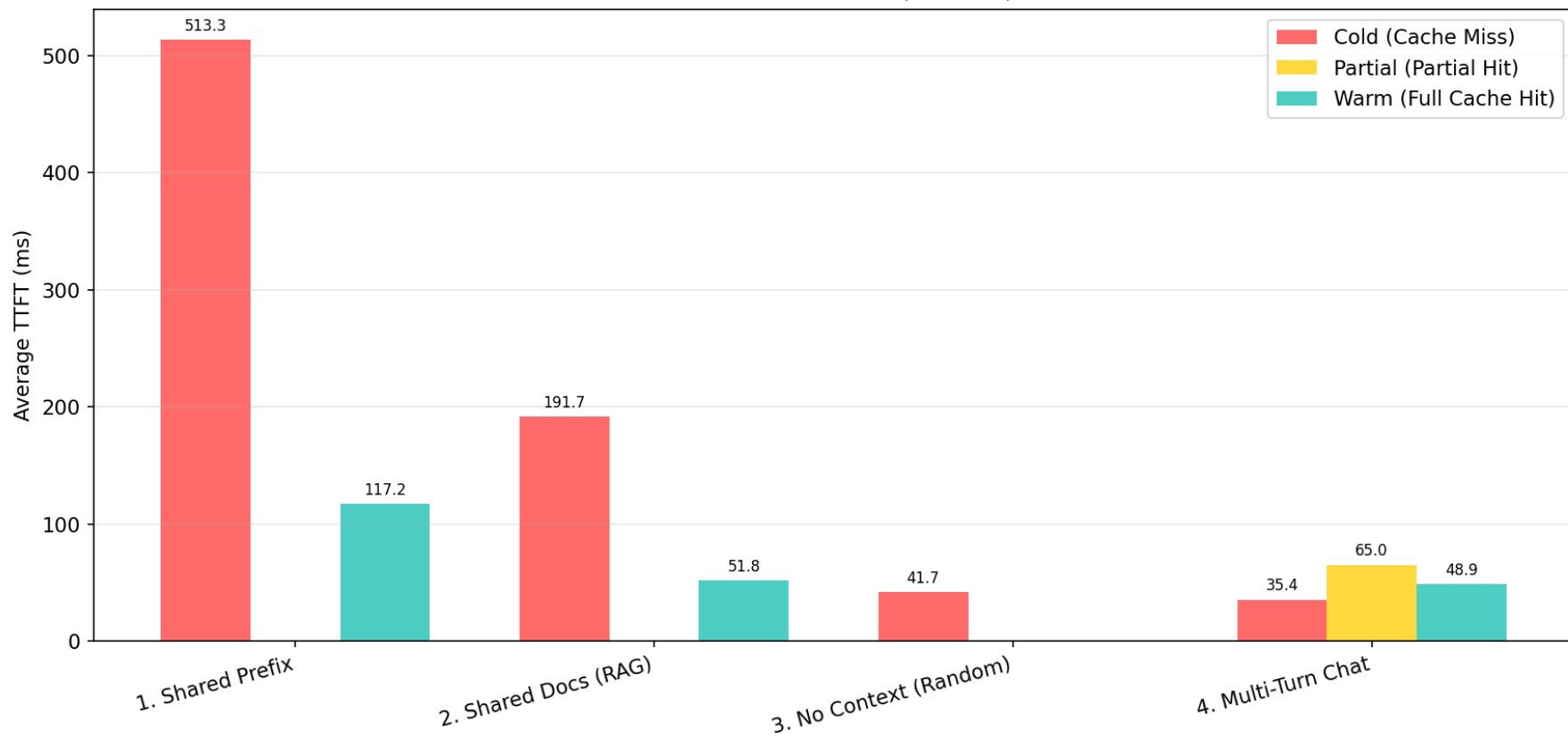
Cold vs Warm Latency by Experiment

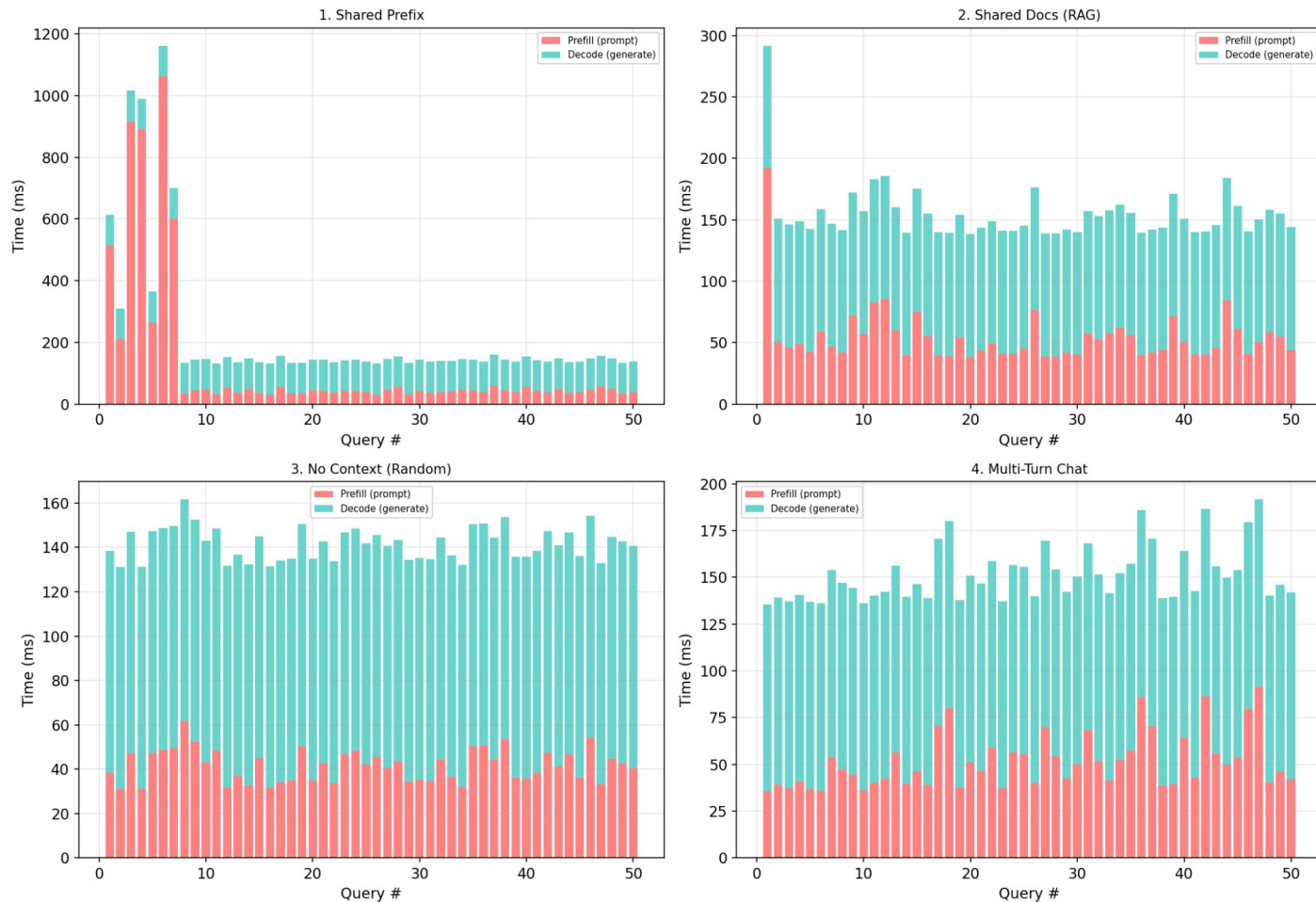


Cold / Partial / Warm Latency by Experiment

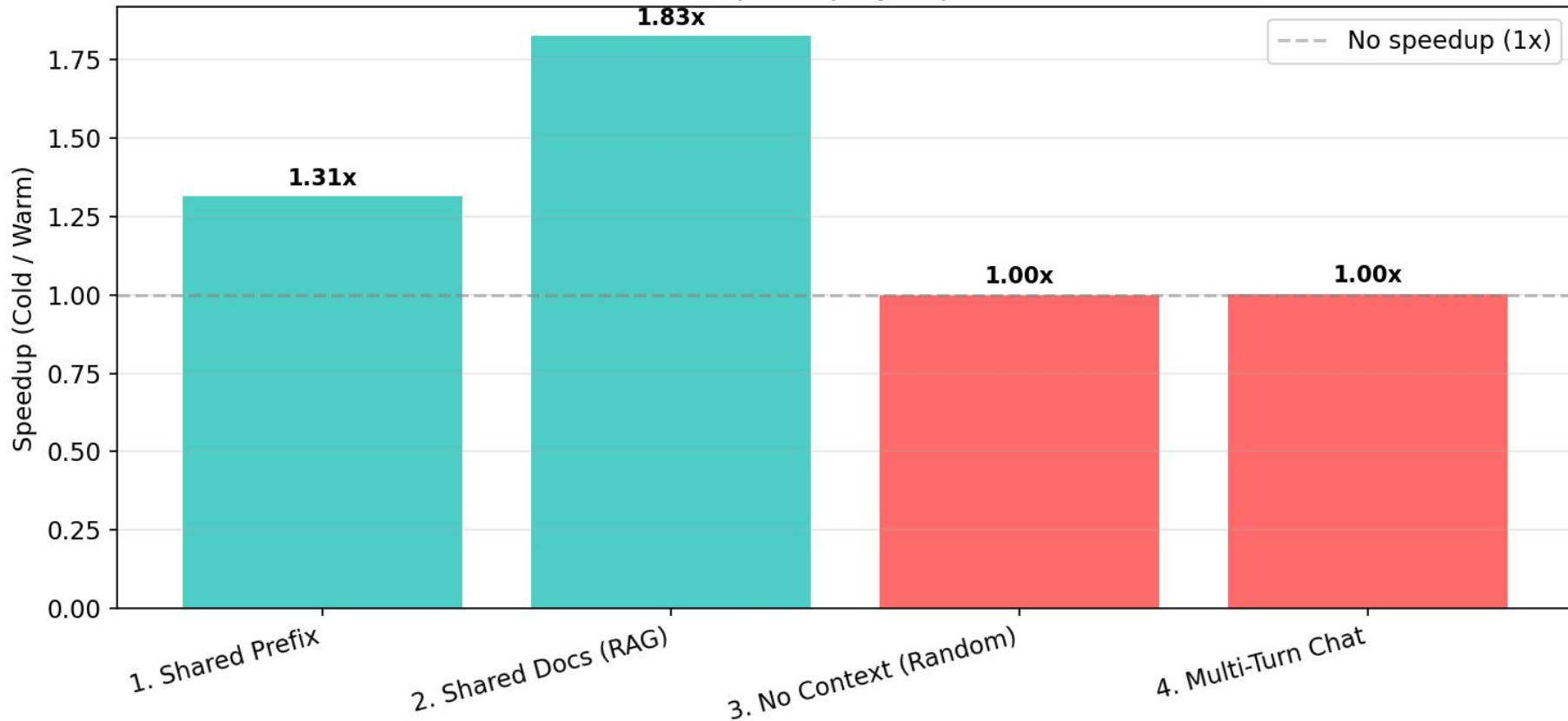


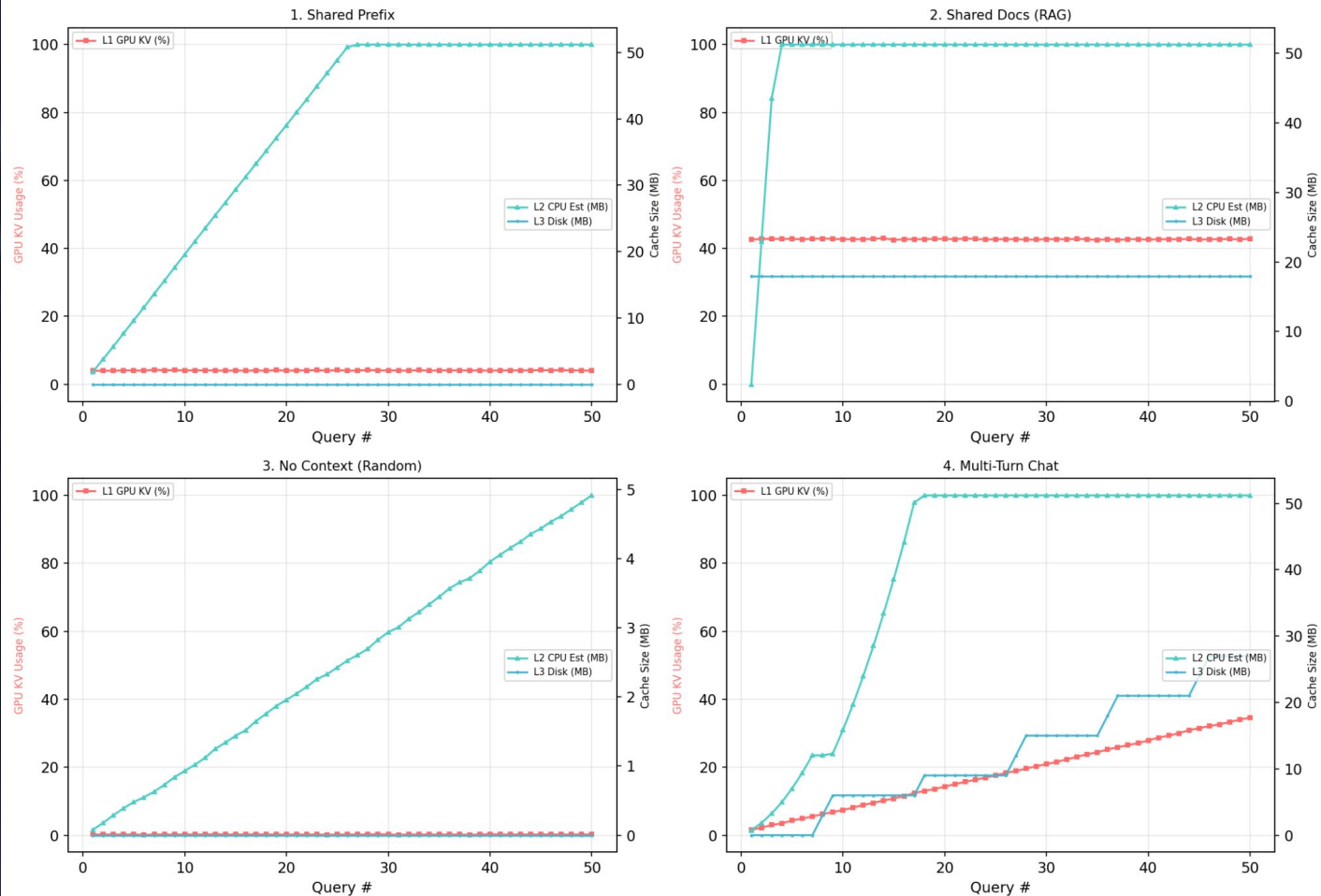
Time To First Token — Cold / Partial / Warm

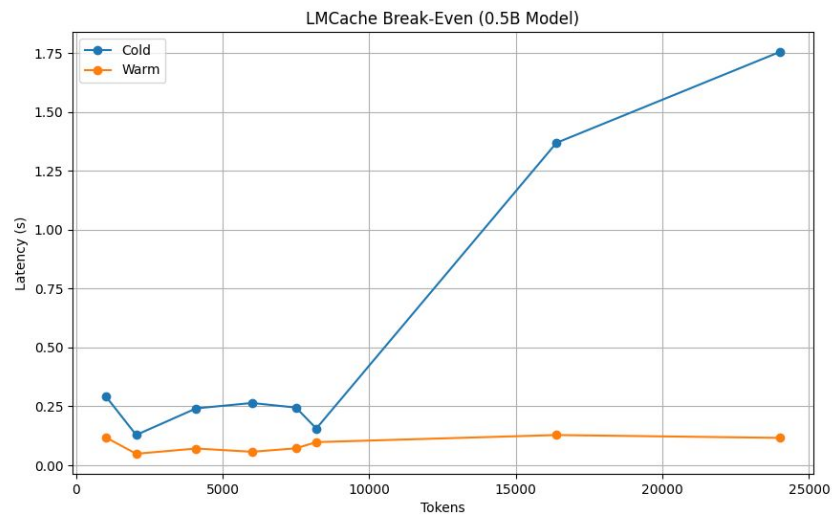
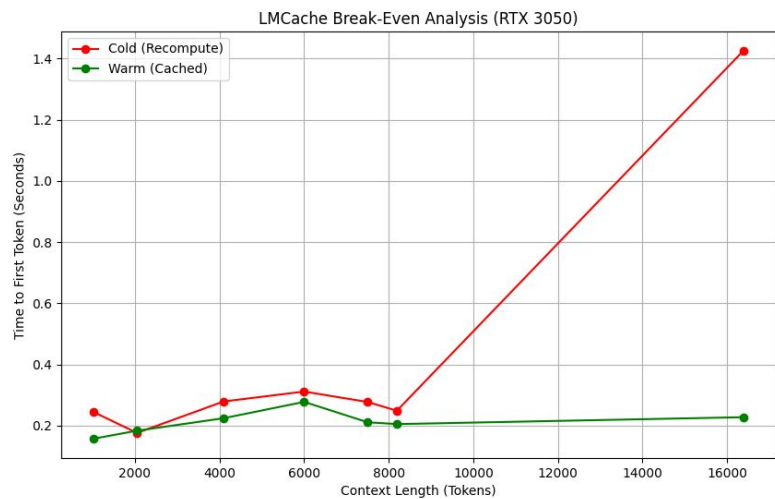




Cache Speedup by Experiment







What we learnt

So, the logical step is to know what part of the cache will be needed, so that you can preload the KV pairs from the cache . Now the problem with this is that we don't know what the next prompt will be.

User prompts also don't necessarily follow the principles of temporal or spatial locality since the next query or question can be from any part of the existing conversation or a totally new question.

How do we deal with this?

Next Steps

Milestone 3: Neural Cache Prefetcher

Instead of waiting for a cache miss, we aim to predict which `KV-cache` chunks will be needed based on history of prompts.

- Generate an “access sequence” trace by logging which `CACHE_CHUNKS` were requested during a session.
For e.g. (`Session_ID`, `Timestamp`, `Current_Query_Embedding`, `Requested_Chunk_ID`)
- Use a `Sentence-Transformer` (e.g. `all-MiniLM-L6-v2`) to convert the input prompt into a vector for the embedding layer.
- Use an `RNN` (or `LSTM`) which takes a sequence of last 3-5 query embeddings and output a probability distribution over available KV-cache chunks in `L3`.
- Based on the prediction, we aim to migrate the required KV-cache data into `L2` before the LLM finishes the prefill phase.

Milestone 4: Verification & Decisions

- Take Decisions like how far to predict (risk vs accuracy), verifying migration waste (measuring `throughput_predictive` vs `throughput_baseline`). If the ratio < 1 : ML model is too heavy / inaccurate
- Performance Benchmarking:
 - Evaluate metrics like TTFT against Sequence Length (session size).
 - Calculate Hit Rate curve & comparing it with normal LRU cache or LM cache with “Neural-Prefetch” hit rate.
 - Calculate real-time inference time of the LSTM prefetcher and determine its effectiveness.
- Generating a final report of the findings and experiments.



Questions?

The background of the slide is a dark blue gradient with numerous thin, flowing, light blue lines that create a sense of movement and depth. These lines are concentrated in the lower half of the image, where they appear to originate from a point and fan out, while the upper half is more uniform in color.

Thankyou